

COMPUTER SCIENCE TRIPOS Part IB – 2023 – Paper 6

9 Semantics of Programming Languages (nk480)

The relational algebra is a small language for manipulating sets of tuples, and is one of the central objects of study in database theory. We can give a syntax for (a subset of) it as follows:

τ	$::=$	$\text{int} \mid \text{bool}$	Data types
d	$::=$	$n \mid b$	Data values
R	$::=$	$[l_1 : \tau_1, \dots, l_n : \tau_n]$	Record types (with disjoint field names l_i)
r	$::=$	$[l_1 = v_1, \dots, l_n = v_n]$	Record values (with disjoint field names l_i)
S	$::=$	$\text{Set } R$	Set types
e	$::=$	$\{r_1, \dots, r_n\}$	Set literal
		$e \cup e'$	Set union
		$e \times e'$	Cartesian product with disjoint field labels
		$\Pi_{l_1, \dots, l_n}(e)$	Records of e with fields not in $l_1, \dots, l_n$ removed
		$\sigma_{l_1=l_2}(e)$	Subset of e where the fields l_1 and l_2 are equal

- (a) State the form of the typing judgements for this language, and give typing rules for this programming language ascribing to each category of terms its corresponding types. [8 marks]

Answer:

$$\begin{array}{c}
 \overline{n : \text{int}} \quad \overline{b : \text{bool}} \quad \overline{[l_1 = v_1, \dots, l_n = v_n] : [l_1 : \tau_1, \dots, l_n : \tau_n]} \\
 \frac{r_1 : R \dots r_n : R}{\{r_1, \dots, r_n\} : \text{Set } R} \quad \frac{e_1 : \text{Set } R \quad e_2 : \text{Set } R}{e_1 \cup e_2 : \text{Set } R} \\
 \frac{e : \text{Set } [l : \tau, l' : \tau, \dots]}{\sigma_{l=l'}(e) : \text{Set } [l : \tau, l' : \tau, \dots]} \quad \frac{e : \text{Set } [l_1 : \tau_1, \dots, l_n : \tau_n, \dots]}{\Pi_{l_1, \dots, l_n}(e) : \text{Set } [l_1 : \tau_1, \dots, l_n : \tau_n]} \\
 \frac{e_1 : \text{Set } [l_1 : \tau_1, \dots, l_m : \tau_m] \quad e_2 : \text{Set } [l'_1 : \tau'_1, \dots, l'_n : \tau'_n]}{\frac{l_i \neq l'_j \text{ for any } i \in [1 \dots m], j \in [1 \dots n]}{e_1 \times e_2 : \text{Set } [l_1 : \tau_1, \dots, l_m : \tau_m, l'_1 : \tau'_1, \dots, l'_n : \tau'_n]}}
 \end{array}$$

Note that there are actually three judgements being defined: $d : \tau$ for data values, $r : R$ for records, and $e : S$ for relations. Also note that there is no context, because this language does not have variables.

To be perfectly correct, the rules for the select and project primitives should technically allow the labels to occur anywhere in the list, in any order. Being precise about working up to permutation is necessary for a fully rigorous proof, but I explicitly do not want to require students to lay all this out.

- (b) Define a deterministic small-step operational semantics for this language, defining any auxiliary functions you need as well. [10 marks]

Answer: First, we define the values and evaluation contexts of the language:

$$\begin{aligned} v &::= \{r_1, \dots, r_n\} \\ E &::= \square \cup e \mid v \cup \square \mid \square \times e \mid v \times \square \mid \Pi_{\vec{l}}(\square) \mid \sigma_{l=l'}(\square) \end{aligned}$$

Now, we can use this to define the operational semantics.

$$\frac{e \rightsquigarrow e'}{E[e] \rightsquigarrow E[e']}$$

$$\overline{\{r_1, \dots, r_m\} \cup \{r'_1, \dots, r'_n\}} \rightsquigarrow \overline{\{r_1, \dots, r_m, r'_1, \dots, r'_n\}}$$

$$\overline{\{r_1, \dots, r_m\} \times \{r'_1, \dots, r'_n\}} \rightsquigarrow \overline{\{r''_1, \dots, r''_{m+n}\}} \text{ where } r''_{i+m \cdot (j-1)} = r_i \bullet r'_j$$

$$\overline{\Pi_{\vec{l}}(\{r_1, \dots, r_n\})} \rightsquigarrow \overline{\{\hat{\Pi}_{\vec{l}}(r_1), \dots, \hat{\Pi}_{\vec{l}}(r_n)\}}$$

$$\overline{\sigma_{l=l'}(\{r_1, \dots, r_n\})} \rightsquigarrow \overline{\hat{\sigma}_{l=l'}(r_1) \cup \dots \cup \hat{\sigma}_{l=l'}(r_n)}$$

We define the auxilliary functions used in this semantics as follows:

$$[l_1 = v_1, \dots, l_m = v_m] \bullet [l'_1 = v'_1, \dots, l'_n = v'_n] = [l_1 = v_1, \dots, l_m = v_m, l'_1 = v'_1, \dots, l'_n = v'_n]$$

$$\hat{\Pi}_{l_1, \dots, l_n}([l_1 = v_1, \dots, l_n = v_n, \dots]) = [l_1 = v_1, \dots, l_n = v_n]$$

$$\hat{\sigma}_{l=l'}([l = v, l = v', \dots]) = \begin{cases} \{[l = v, l = v', \dots]\} & \text{when } l = l' \\ \{\} & \text{otherwise} \end{cases}$$

Note that the latter two definitions are defined only up to permutation of the label fields. Being precise about this is necessary for a fully rigorous proof, but I explicitly do not want to require students to lay all this out.

- (c) Give a precise statement of the progress and preservation properties for this language. You do not need to give a proof. [2 marks]

Answer:

- If $e : S$ then either e is a value or $e \rightsquigarrow e'$ for some e' .
 - If $e : S$ and $e \rightsquigarrow e'$ then $e' : S$.
-